

DEPARTMENT
OF
COMPUTER ENGINEERING

CNG 351
Data Management and File Structures
Assignment 2
(6% of actual grade)

DUE DATE: 18 November 2023, 23:55 (Cyprus Time).

IMPORTANT RULES

1. Use a graphical tool such as Lucidchart or Draw.io.
 - Lucidchart can be used to design and create your solution. You can find a full tutorial on ER diagrams and Lucidchart here:
<https://www.lucidchart.com/pages/er-diagrams>.
Please make sure that you use **Chen notation** for your design, other notations will not be accepted.
For Lucidchart, please ensure that you have an educational account. **To get an educational account, you need to sign up with your METU mail addresses from the following link:**
<https://lucid.app/pricing/lucidchart>
 - Draw.io can also be used to draw your diagram. You can find the full details here: www.draw.io/.
2. When designing with EER, any assumptions must be explicitly explained. If your assumptions do not make sense in the given requirements content or conflict with the requirements or similarly, if you assume and did not specify the assumption, you will lose points from that part of your design.
3. For logical mapping, you need to use the notation detailed in the Part 2 in page 7 for logical representation of a database. Please use that notation to represent relations, PKs (primary keys) and also FK (foreign keys).
4. Create a PDF file for your solution, and upload only one PDF file to ODTUClass, one team member is enough to upload the solution.
5. Assignments will be completed by a team. Therefore, for this first assignment please make sure that you form a team and continue to work with that team throughout the semester.
6. Please submit a report that includes a cover page with the team details including their names/surnames and also student IDs.

PURPOSE

This assignment has two purposes: (1) to help you revise EER concepts and (2) to help you revise how to map an (E)ER design to a logical design. Therefore it has two parts.

GRADING

This assignment has two parts and each part will be 50 percent. Each part has two use cases and therefore, the overall grading will be as follows:

1. Part 1: Use Case 1: 25 points;
2. Part 1: Use Case 2: 25 points;
3. Part 2: Use Case 1: 20 points;
4. Part 2: Use Case 2: 25 points;
5. Report: 5 points. A good report means typewritten, complete (every section fulfilled), clear explanations in English (where relevant), and submitted via ODTUCLASS as one combined PDF document. The report's first page must be a title page which should clearly state team details and assignment number. Each use case should also include Assumptions written. The footer of all subsequent pages should be numbered in the format x of y (eg 2 of 6), etc.

PART 1: USE CASES

The overall goal of this part is to help you revise EER concepts. In the previous assignment, we gave you four use cases and in this assignment, we are continuing with two of those which now have more detailed requirements. New requirements added are all underlined. You need to take your design from the previous assignment and extend it to meet the needs of the newly added requirements and design them with EER. Please note that some of the previous requirements need to be also revised to encode them with EER. You should not just focus on the new, underlined requirements.

Use Case 1 - ACM Fest Management System

METU NCC ACM Student Society organizes a hackathon annually in the name of "ACM Fest". The society developed an online ACM Fest registration system to organize everything and create an archive for every year's submissions and details. Anyone participating in these annual hackathons, including supervisors and judges, should use the system.

- The participants should register in the system before assembling a team. They must enter their email, name and surname, department, date of birth, and university. Department and university information is recorded as all participants will be university students. After creating a participant profile, they can have three different roles: Team Leader, Team Member, and Visitor. Team Leaders are responsible for forming and leading the team. One of the team members is the leader. A student can lead only a single team at a time, but ofcourse they can create different teams over the years. Team Members are responsible for participating in a team in the hackathon. The system will record every years team and team members. Finally, participants can become visitors to the side-event of that year's hackathon, the summit. A participant can become a visitor

and participate in the summit without joining a team. But of course, the team members can attend the summit as well.

- In the hackathons, the team can include between one to five participants. Every team has an ID, a name, a watchword, the university they are representing, and the established date.
- For professionals such as academics and industry veterans, registration is a little different. the ACM Fest team will send the special registration link to the academics and industry veterans if they want to become judges or supervisors. They will need to enter their name, email, gender, profession, date of birth, and their institute (company/university) After registration, they can choose how to participate in that year's hackathon. They can either become a judge or a supervisor, but can not become both in the same hackathon. Despite that, all of the professionals can become a speaker at that year's summit. Note that a professional can register just to attend the summit.
- Teams can add their supervisor (teams only have one supervisor) if they have been registered. Supervisors can supervise multiple teams at the same time, and the system should keep track of their list of research areas.
- After the registrations and team creations are done, teams will add their projects for that year's hackathons into the system. A project can not exist without a team. Team members will add their project name and description to the system. Note that project names can be the same within different years from different teams, but the same team can not use the same project name. Every team can participate in only one project per year, but if they don't disband the team, they can create another project for the next year's event.
- During the hackathon, the projects will be evaluated by that year's judges. Judges will evaluate the submitted projects of that year's hackathons with a score out of 100. At least four judges evaluate a project. However, every judge is responsible for evaluating every single project that participated. The final scores of the projects will be the average of every score the judges gave, and those scores will be recorded.
- The system also will keep track of the hackathons since every year's event is different. Every hackathon will have its ID, year, theme, and duration. Please note that the system needs to record which projects are submitted for which hackathon. Plus, every year, there can be only one winning project in that year's hackathon. The winning project also needs to be recorded.
- Finally, Every year's hackathon will include a summit to bring the students and professionals together. Summits are a part of the hackathon and they will be organized during the ACM Fest. The system will keep the date and description information for summits, and it will record every student and professional who participates in the summit. Also, the ACM society wants to ensure a "Certificate of Participation" for the students who attended the summit. So, a certificate hash for every student per summit will also be kept in the system.

Use Case 2 - Online Tutoring System

In Cyprus, a smart online tutoring system was developed to address the growing demand for students for flexible and additional academic support offered by tutors who specialize in various subjects. This platform connects students with the tutors in a way that allows them to conduct private tutoring sessions either for supplementary help in their ongoing courses or to gain proficiency in a new subject. The system integrates functionalities other than connecting students with tutors, such as progress tracking, course and lecture management, and secure payment processing.

The system has different users, including students who take classes, tutors who provide their expertise and tutoring services, and admin who manage tutor registrations and ensure quality. For any of them to access the system, they need to register on the platform and provide personal information such as their email, name, surname, and role. The role can be student, tutor or admin, and the system will automatically generate a unique ID for each user. If 'student' is chosen as a role, then the user needs to enter their phone number and the university that they are currently studying. However, if the user registering is a tutor, then the system will ask them to enter the areas of specialization. The profiles of the tutors allow students to search and select tutors who meet their specific learning needs. Tutors should also have a rating that is provided by students after each lecture. This ensures the quality of the tutoring experience. The system has two types of tutors: short term and long term tutors. Each tutor teaches several courses that have a unique ID, a description, and a name, recorded in the system. For example, a possible course would be C-Course or Java-Course. Moreover, once a course is added by the tutor, they should add the lectures associated with that course. For example, a lecture would be from Monday 10-11.

Courses can also be specialized as; Supplementary courses, Proficiency courses, or Exam Preparation courses. Supplementary courses are courses that provide support for ongoing academic classes. Each supplementary course should have to store the school subject of the student that this course relates to. Proficiency courses are focused on building proficiency in a new subject. When viewing this course, the students should be able to see the Proficiency Level of the course (Beginner, Intermediate, Advanced), and whether it provides a certification or not. The last type of course is the Exam Preparation course, which is designed and focuses on a specific type of exam such as SAT, TOEFL, GRE, IELTS, etc. Please note that long-term tutors can teach all kinds of courses, however short term tutors can only teach exam preparation courses.

Once a student and tutors register to the system, students can view the profiles of the registered tutors, their courses, and their related lectures. They will also be allowed to get feedback on their progress from the tutors after each lecture, and provide feedback or rank their tutors based on their learning experience after the lecture.

Once students book a lecture, they are assigned a lecture session that a tutor has

already scheduled for a specific course. Each lecture has an identified cost, date, start time, and end time. Moreover, the system ensures secure payment handling and records all completed transactions. Thus, once a lecture is booked, a payment process is initiated, where students must pay using various methods. The payment has a unique ID and records of the date, time, amount, and payment method.

As mentioned earlier, when the lecture finishes, tutors are responsible for generating and sending progress reports to the students. These reports provide detailed feedback on the student's performance and progress. Students, in return, receive these reports, which allows both the student and the tutor to track the improvements over time. In these reports, tutors record the feedback that they want to give to the students, for example, if the student was able to follow the lecture and if they were able to understand the lecture, the system also records the date and time of the report. Similarly, after each lecture, students can rate their tutors based on their performance. For ratings, the system uses a rating between 1-10 (1: bad, 10: excellent). These ratings are reflected in the tutors' profiles so that they help other students select tutors; it also encourages the tutors to deliver high-quality education and improve their skills based on students' evaluations. In the tutor profiles, the system calculates the averages of all the ratings they receive for all their lectures. Therefore, if a tutor teaches two courses with two lectures each, their overall rating will be the average of all four lectures.

Furthermore, the system has a number of administrators who manage the registration of the tutors; their job is to check the tutor's credentials. This ensures that the tutors on the system are well-qualified and provide quality tutoring. A tutor applies and once they are approved, then the system records who approved a tutor. For admins, please note that the system records, their ID, username, name/surname, and email.

PART 2 – USE CASES

When you do the logical mapping, in your report please use the following notation illustrated with the following example:

Employee

<u>ssn</u>	name	surname	<u>dept_id</u> [FK: Department: dept_id]
------------	------	---------	--

Department

<u>dept_id</u>	name	address
----------------	------	---------

Notation Notes:

- Primary Key: represented with underline text;
- Foreign Key: represented with dotted underline text and the reference is represented with [FK: TargetRelation:targetRelationKey]

Use Case 1 – METU NetRegister

At METU NCC, students must register their devices to our campus network registration system to gain access to the campus network. The university has implemented a comprehensive network registration system called METU NetRegister to manage device registrations and facilitate this process. The main goal of this application is to store information about students, devices, registrations, network engineers, and related network issues and keep track of the registered devices' VPN connection details. Their database designers provided a design for the requirements – see Figure 1. You need to map this design to a logical one that can be implemented. Please note that this is not directly created from the requirements given to you in the previous assignment – some changes have been made to the requirements given to you.

Use Case 2 – Desire for Velocity

Desire For Velocity is a multiplayer competitive racing game developed by some brilliant indie game developers in METU NCC. It's a real-time racing game with different cars and car classes, races with matchmaking, tournaments, and single-player drift mode. These developers designed the data requirements which is given in Figure 2. You now need to map this to a logical design that can be implemented. Please note that the requirements encoded in this figure are not the direct requirements given to you in the previous requirements – they have been significantly updated and changed.

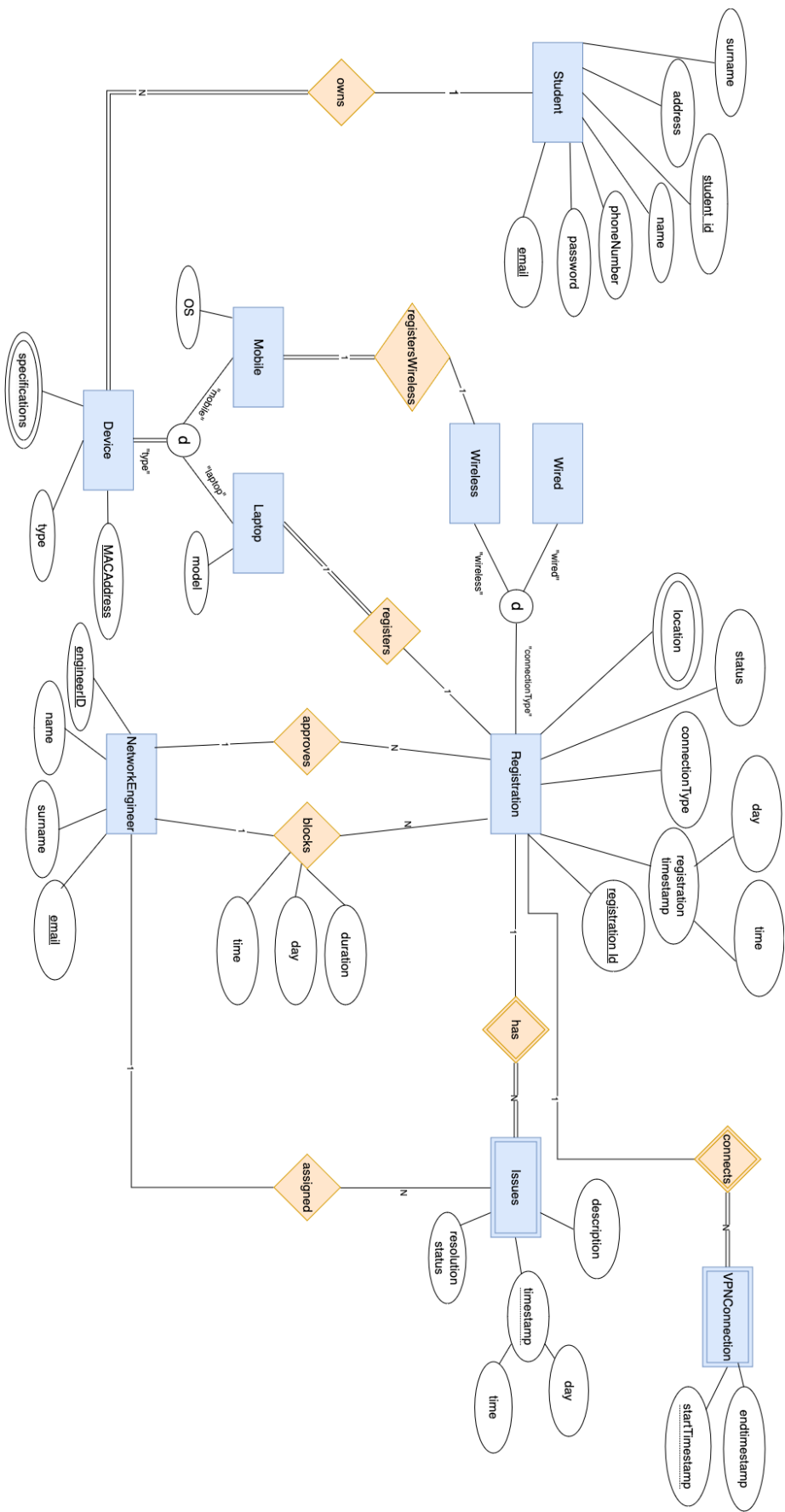


Figure 1: EER Diagram for Use Case 1

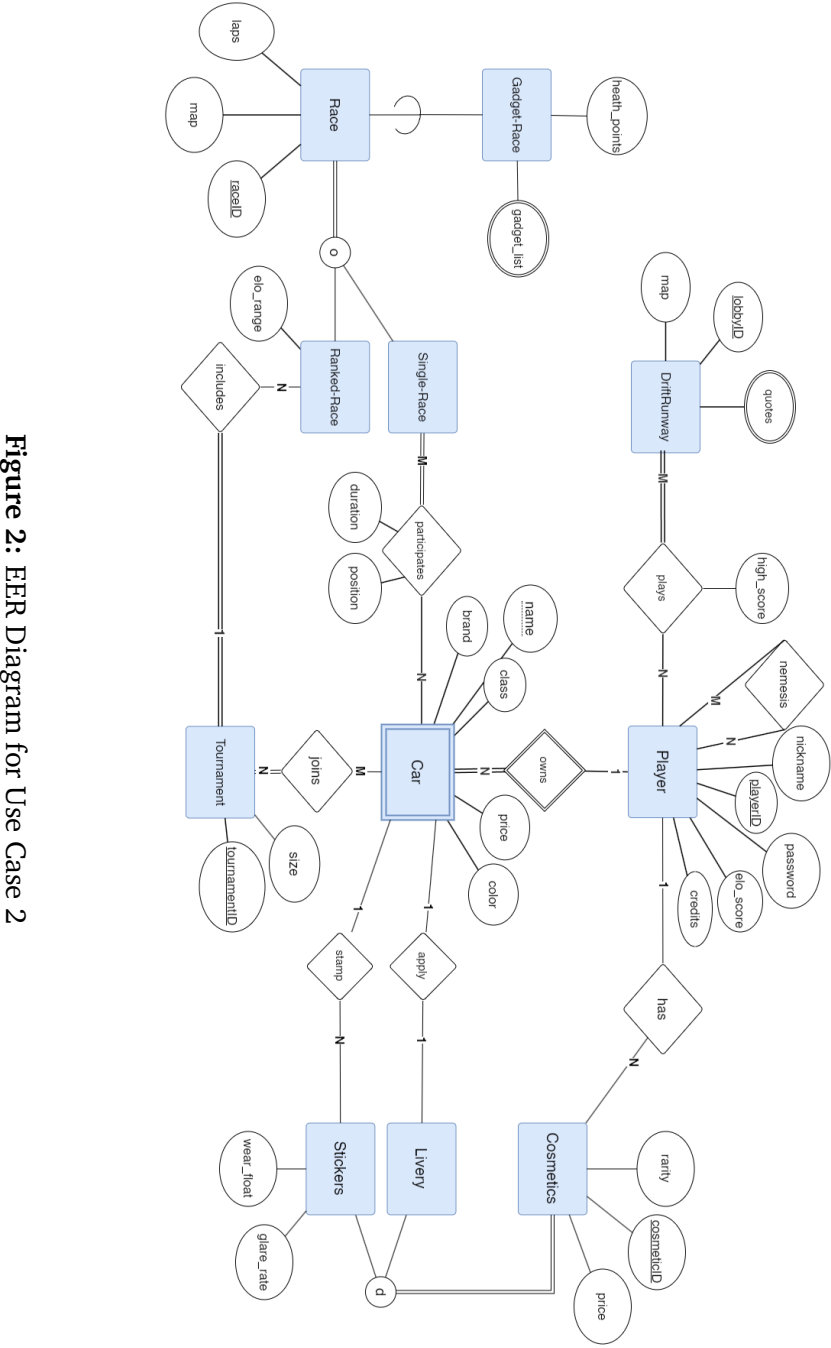


Figure 2: EER Diagram for Use Case 2